Vietnam National University Ho Chi Minh City
Ho Chi Minh City University of Technology
Faculty of Computer Science and Engineering

THESIS PROPOSAL – COURSE: CO5109

Cutting the Cost of LLM-Based Program Repair with a Cost-Aware Validation Layer



Faculty: Computer Science and Engineering
Semester: 261
Instructor: Dr. Le Xuan Bach, CSE-HCMUT
Student: Nguyen Thai Thanh Dat
Student ID: 2570387

Ho Chi Minh City, November 2026

Abstract

In LLM-based program repair, each round has two costs: the model call that proposes a patch and the test run that checks it. Part of this cost is wasted. The model often repeats patches it has already proposed, a wrong patch may pass many tests before one fails, and sessions continue after the model has stopped making progress. The size of these wastes has not been measured.

This proposal builds a validation layer, placed between the repair loop and the project's test runner, with one module aimed at each waste. A *Repeat Guard* recognises repeated patches and reuses their results. A *Test Prioritiser* runs first the tests most likely to fail per second of running time, using a rule, Arc, that learns both quantities during the session. A *Budget Guard* stops sessions that are stuck or out of budget and reports why. The layer is designed for a repair bot in continuous integration and needs no instrumentation. The proposal also measures how the cost of a repair loop is distributed, to know how large each waste is and so the most each module can save.

The evaluation on 100 Defects4J bugs (50 for the end-to-end run) has two levels. A simulation on mutated patches, which every test ordering sees alike, compares Arc with Frtp and with the rule of the mutation tool PIT, with and without first running the failing tests. Live runs with a real model then measure where the cost goes and whether the whole layer lowers the cost per repaired bug without repairing fewer bugs. The test ordering is expected to save mainly on validation; model latency and build time dilute its effect end to end.

Keywords: automated program repair, large language models, patch validation, test prioritisation, cost analysis

Contents

Abstract **ii**

1 Introduction **1**
1.1 Problem Statement . . . 1
1.2 Research Questions and Contributions . . . 2
1.3 Scope . . . 3
1.4 Structure . . . 3

2 Background **4**
2.1 APR Pipeline . . . 4
2.2 Cost Model of a Repair Session . . . 4
2.3 Test Ordering under Early Stopping . . . 5
2.4 Regression Test Prioritisation . . . 7
2.5 Censored Test Outcomes . . . 7
2.6 Benchmark . . . 9

3 Related Works **10**
3.1 LLM-Based Repair Systems . . . 10
3.2 Validation in Current Tools . . . 11
3.3 Cost Measurement in LLM-Based Repair . . . 12
3.4 Classification of Cost-Reduction Techniques . . . 12
3.5 Test Prioritisation . . . 12
3.6 Other Cost-Reduction Techniques . . . 14
3.7 Research Gap . . . 15

4 Proposed System **17**
4.1 Usage Scenario and Requirements . . . 17
4.2 System Overview . . . 17
4.3 Module M1: Repeat Guard . . . 19
4.4 Module M2: Test Prioritiser . . . 19
4.5 Module M3: Budget Guard . . . 22
4.6 When the Layer Helps . . . 23

5 Evaluation Plan **24**
5.1 Bugs and Setup . . . 24
5.2 What Is Compared . . . 25
5.3 How Cost Is Measured . . . 25
5.4 E1: Orderings on Mutants . . . 26

5.5 E2 and E3: The Loop, Live 27
5.6 Metrics 29
5.7 Statistics 29
5.8 Pilot Study 30
5.9 Compute Budget 30
5.10 Pre-Registration 30

6 Expected Outcomes **32**

7 Work Plan and Conclusion **34**
7.1 Work Plan 34
7.2 Risks 34
7.3 Threats to Validity 35
7.4 Future Work 35
7.5 Conclusion 36

References **37**

Chapter 1

Introduction

1.1 Problem Statement

Context. Automated program repair (APR) fixes a faulty program without a human writing the patch [1]. Most tools work in a loop: they generate a candidate patch and run the test suite on it; a candidate that passes every test is called *plausible* [2]. Today the candidate comes from a large language model (LLM). If a test fails, the tool shows the failure to the model and asks again [3]. Each round of this loop has two costs (Figure 1.1): the model call and the test run.

Need. The cost per bug is now reported next to the repair rate: ChatRepair spends about $0.42 per bug [3] and RepairAgent about 270,000 tokens per bug, with 99 % of its time spent running tools, mostly tests [4]. Tools already try to lower this cost [5], and long, costly sessions are named as an obstacle to using LLM-based repair in practice [6]. The practical need is therefore to *lower the cost per repaired bug without repairing fewer bugs.* A typical setting is a repair bot in continuous integration (CI), which is where developers most want repair to run [7]. Such a bot must respond within a bounded time and budget, fit into the existing pipeline, never propose a patch that the tests reject, and explain its failures.

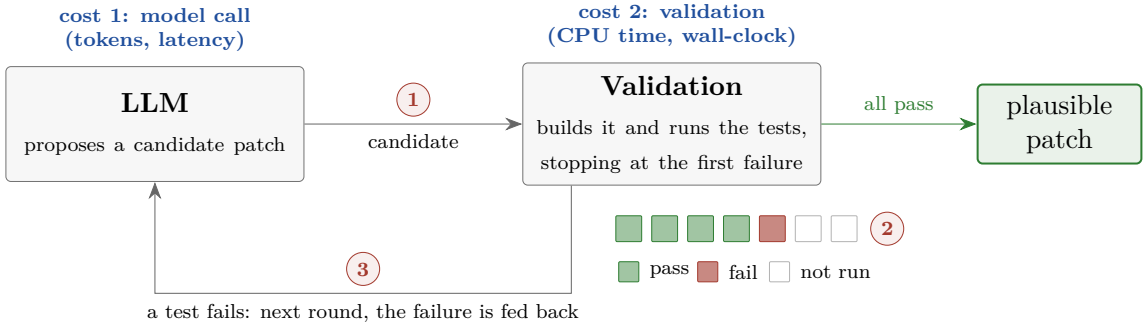


Figure 1.1: One round of an LLM-based repair loop and its two costs. Under the validation step, the tests of a rejected candidate: passing tests (green) run before the failing one (red), and the rest are not run. The marks are the three wastes: ① a repeated candidate is generated and tested again; ② passing tests run before the one that rejects a wrong candidate; ③ rounds go on after the model has stopped making progress.

Problem. A large part of this cost is spent without benefit, in three ways:

- **Repeats.** The model proposes patches it has already proposed: 65 % of the patches of ChatRepair were duplicates [8]. Each repeat costs a model call and often a test run.
- **Slow rejections.** One failing test is enough to reject a wrong patch, but a fixed test order may run many passing tests first. Test orderings for repair exist [9], but they ignore how long each test takes.
- **Stuck sessions.** A session goes on after the model has stopped making progress, until its budget runs out. These failed sessions are the expensive ones [10].

The size of each waste is unknown, because existing studies report either the generation or the validation cost, for different tools and settings. It is therefore not known how much a technique aimed at each waste can save at most.

Goal. The three wastes are identified from the literature. This thesis builds a *validation layer*, a component placed between the repair loop and the test runner, with one module aimed at each waste:

1. **Module M1, Repeat Guard**, avoids validating the same patch twice: it recognises a repeated patch, reuses its result and informs the model;
2. **Module M2, Test Prioritiser**, rejects wrong patches early: it runs first the tests with the highest probability of failing per second of running time;
3. **Module M3, Budget Guard**, stops unproductive sessions: it ends a session when the model makes no progress or the budget is spent, and reports the reason.

The first two never change whether a patch is accepted. The third can, so its effect on the repair rate is measured. The thesis also measures how the cost of a repair loop is distributed, to know how large each waste is, that is, the upper bound on what the module aimed at it can save.

1.2 Research Questions and Contributions

- **RQ1: Where does the cost of an LLM-based repair session go?** The cost per repaired bug, split into generating and validating patches, and how much of it falls after the model is stuck. The size of each waste bounds what the module aimed at it can save in RQ2.
- **RQ2: How much does each module save?** Each of the three modules is measured on the part of the cost it targets. The test ordering, Arc, is compared with Frtp [9] and with the coverage-based rule of the mutation tool PIT [11].
- **RQ3: Does the whole layer lower the cost per repaired bug?** Against a standard conversational loop, without a lower repair rate.

- **RQ4: Why does Arc differ from existing orderings?** Which of its parts matters, and when.

Contributions.

C1 **A cost breakdown of LLM-based repair loops**, with the most each kind of technique could save.

C2 **A cost-aware validation layer** of three modules: a repeat guard, a test prioritiser and a budget guard.

C3 **Arc** (Adaptive Rate-over-Cost), the test ordering at the core of the prioritiser: it learns, during the session, how likely each test is to fail and how long it takes, and needs no extra instrumentation.

C4 **A two-level evaluation** on Defects4J: a simulation in which every ordering sees the same patches, and live runs with a real model.

C1, C3 and C4 are the main research contributions. The repeat guard and the budget guard are included mainly for practical reasons, since they are simple, do not affect correctness and are absent from current tools (Section 3.2).

1.3 Scope

Table 1.1: Scope of the study.

In scope	Defects4J [12] (Java), 100 single-file bugs; one LLM through an API; a conversational loop with test feedback; the faulty methods are given to the model; success means a plausible patch
Measured	time, CPU time, tokens and money
Out of scope	other languages and benchmarks; finding the faulty code; judging whether a plausible patch is correct; tests run in parallel on many machines

1.4 Structure

The rest of the proposal is organised as follows.

- Chapter 2 gives the background: the cost of a repair session and how to order tests.
- Chapter 3 reviews related work and the open problems.
- Chapter 4 presents the validation layer and its three modules.
- Chapter 5 plans the evaluation.
- Chapter 6 states how each possible result will be read.
- Chapter 7 gives the work plan, risks, threats to validity and future work.

Chapter 2

Background

2.1 APR Pipeline

Figure 2.1 places validation in the full repair pipeline. Fault localisation finds the code most likely to hold the bug. The generator proposes a candidate patch there, and the validator builds it and runs the test suite. A plausible candidate goes on to a human or automatic correctness check; a failing one, in an LLM-based loop, returns its failure to the model, which proposes the next candidate.

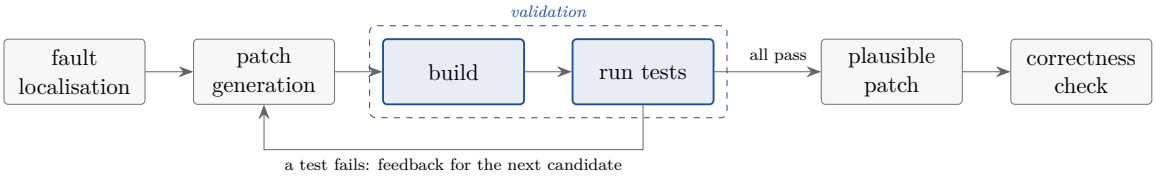


Figure 2.1: The generate-and-validate pipeline. Validation (highlighted) runs once per candidate.

Part of each validation is overhead (recompiling and starting the runtime), which on the JVM can be removed by validating in one long-lived JVM [13, 14]; the rest is running tests.

2.2 Cost Model of a Repair Session

This section makes precise the first question of the thesis: where the cost of a repair session goes. A session runs rounds $i = 1, \ldots, R$; in each round the model proposes one candidate p_i and the validator judges it. The session ends at the first plausible candidate or when the round budget $R_{\max}$ is spent. Its cost is

$$E = \sum_{i=1}^{R} G_i + \sum_{i=1}^{R} V_i, \tag{2.1}$$

where G_i is the cost of the model call of round i and V_i everything it costs to validate its candidate: applying the patch, building it, starting the runtime and running the tests. Cost is measured in three units, each reported on both sides: wall-clock time, which is what a user waits for; compute (CPU seconds for validation, tokens for generation); and money, from a fixed price list. The model has two layers: what the session spent, and what each family of techniques could have saved.

What the session spent. A candidate is a *repeat* if its normalised text equals that of a candidate already proposed in the same session. Splitting the rounds by whether their candidate is a repeat, and the validated candidates by their verdict, gives five components, each read directly from the session:

$$E = \underbrace{G_{\text{new}} + G_{\text{rep}}}_{\text{generation}} + \underbrace{V_{\text{rep}} + V_{\text{rej}} + V_{\text{acc}}}_{\text{validation}}, \tag{2.2}$$

where G_{new} and G_{rep} are the generation of new and of repeated candidates; V_{rep} is the validation of repeats, which a verdict cache removes; V_{rej} the validation of new candidates that were rejected; and V_{acc} the validation of the accepted candidate, which runs the whole suite in any order.

What each family could save. Each family of techniques (Section 3.4) acts on part of Equation (2.2), and that part bounds what it can save (Table 2.1). Two bounds need a definition. The *ordering headroom* is $V_{\text{rej}} - V^*_{\text{rej}}$, where V^*_{rej} is what rejecting the same candidates would have cost had each been rejected by its cheapest rejecting test, which is the least any order can pay (Section 2.3). The *start-up share* is the part of validation spent building candidates and starting the runtime rather than executing tests. Unlike the five components, the bounds are counterfactual; all of them are computed after the session from the outcome and cost of every test on every distinct candidate, without running any technique.

Table 2.1: What each family of techniques can save at most, in the terms of Equation (2.2).

Family	Upper bound on its saving
Verdict cache (patch reduction)	V_{rep}
Test prioritisation	the ordering headroom, $V_{\text{rej}} - V^*_{\text{rej}}$
Test selection	the cost of tests that do not reach the change; zero where every test runs the whole program
Execution acceleration	the start-up share of validation
Generation-side reduction	G_{rep}, plus the validation of the rounds saved

The share of each component in E is not a property of a technique but of a setting. The generation side scales with the latency and price of the model; the start-up share with the runtime and the build; the rest of validation with the number of tests, their cost and its spread. Two consequences follow. A technique can save a large fraction of its own component and a small fraction of E; and the same technique can be worth deploying in one setting and not in another.

2.3 Test Ordering under Early Stopping

This section states the problem test ordering solves: it concerns the test-execution part of V_i in Equation (2.1). A session produces candidates $p_1, p_2, \ldots$ in order and has a test suite T. For a candidate p and test t, let $o(p, t) = 1$ if t rejects p and 0 otherwise, and

let $c(p,t) > 0$ be the cost of running it (CPU time unless stated otherwise). Neither is known before the test runs. A candidate is plausible when no test rejects it.

Definition 2.1 (Validation with early stopping)**.** For each candidate p, the validator runs the tests in an order σ_p and stops at the first test that rejects p. The order for a candidate may depend on everything observed on earlier candidates.

The validation cost of a session is

$$C \;=\; \sum_{p} \sum_{j=1}^{r_p} c\Big(p, \sigma_p(j)\Big), \tag{2.3}$$

where r_p is the position of the first rejecting test, or $|T|$ if none rejects, and the sum runs over the candidates actually validated: a repeat whose verdict is served from a cache contributes nothing. In the terms of Equation (2.2), C is the test-execution part of $V_{\text{rej}} + V_{\text{acc}}$ with a verdict cache, and of V_{rep} as well without one. A technique is *safe* if it declares a candidate plausible exactly when it is. Reordering is safe whenever tests are deterministic and their outcomes do not depend on the order in which they run, since every test still runs before a candidate is accepted.

Lower bound. If all outcomes were known, the cheapest cost would be

$$C^* \;=\; \sum_{p \text{ invalid}} \min_{t\,:\,o(p,t)=1} c(p,t) \;+\; \sum_{p \text{ plausible}} \sum_{t \in T} c(p,t). \tag{2.4}$$

The second term is fixed: a plausible candidate runs the whole suite in any order. Ordering acts only on the first term, the cost of rejecting invalid candidates. In a session that stops at the first plausible candidate, every candidate but the last is invalid by construction. Search-based tools generate many invalid candidates, so there the first term dominates. An LLM-based loop validates few distinct candidates per session, so the second term (one full suite) can be of the same order as the first, and the room left for ordering is correspondingly smaller. Which term dominates is measured in the experiments (Chapter 5).

Optimal order. The validator does not know the outcomes in advance, so it has to estimate them. If each test rejected independently with probability q_t at cost c_t, the best order is a classical result: the least-cost testing sequence [15], also derived for satisficing search [16]. It is restated here, with the exchange argument, because the rest of the thesis estimates its two quantities.

Proposition 2.2 (Rate-over-cost order [15, 16])**.** Under independent rejection probabilities q_t and costs c_t, running tests in decreasing q_t/c_t minimises the expected cost of rejecting an invalid candidate.

Proof. For adjacent tests a and b, running a first costs $c_a + (1 - q_a)\, c_b$ in expectation and running b first costs $c_b + (1 - q_b)\, c_a$. So a should go first exactly when $q_a/c_a > q_b/c_b$. Any other order contains an adjacent pair whose swap lowers the expected cost. □

Example. Take two test classes. Class A rejects a wrong patch with probability 0.5 and takes 10 seconds; class B rejects it with probability 0.2 and takes 1 second. Running A first costs on average $10 + 0.5 \times 1 = 10.5$ seconds, since B runs only when A passes; running B first costs $1 + 0.8 \times 10 = 9$ seconds. The rule gives the same answer: q/c is 0.05 for A and 0.2 for B, so B goes first even though A fails more often. A failure count would put A first.

A failure count, as in FRTP, estimates neither q_t nor c_t. Proposition 2.2 is the target; the contribution of this thesis is not the rule but estimating its two quantities online, from what validation reveals.

Target rate. When rejections are correlated, what matters for a test at a given position is its rejection probability *given* that every test before it has passed. A record built under early stopping estimates a rate of this kind (rejections among the candidates that reached the test), but averaged over whatever tests happened to precede it, so it equals neither the marginal rate nor the conditional rate of any one order. The thesis takes the rate conditional on reaching the test as the target, and will measure how far the record departs from it (Chapter 5).

2.4 Regression Test Prioritisation

Ordering tests is an old problem in regression testing, where a changed program is re-tested against an existing suite. Rothermel et al. [17] introduced the two greedy strategies that later studies, including the study of validation in repair by Lou et al. [18], use as baselines: *total* ($\mathrm{TP}_{\mathrm{tot}}$) ranks tests by how much code they cover, and *additional* ($\mathrm{TP}_{\mathrm{add}}$) by how much they cover that earlier tests did not. An order is scored by how early it detects faults:

$$\mathrm{APFD} \;=\; 1 - \frac{TF_1 + \cdots + TF_m}{n\,m} + \frac{1}{2n}, \tag{2.5}$$

where n is the number of tests, m the number of faults and TF_i the position of the first test revealing fault i. Elbaum et al. [19] extended it to APFD_C, which weighs each test by its cost.

Two differences separate patch validation from this setting. Under early stopping only the *first* failure of each candidate matters, so the relevant score is the cost paid before it (Equation (2.3)) rather than how early all faults are found. And in a repair loop the "program" changes with every candidate, so an order can learn from one candidate to the next.

2.5 Censored Test Outcomes

Picture the outcomes of a session as a matrix: one row per candidate, one column per test. An order learns from this matrix, but under early stopping it sees only part of

it. For each invalid candidate it observes a few passes and one rejection, and nothing about the tests after the rejection. The record is *censored*, and by construction not at random: a test is observed only when every test ranked before it has passed, so the tests ranked first are observed most often. Four consequences follow.

- **A count is not a rate.** A count of rejections gives zero both to a test never run and to a test run many times without rejecting. A rate (rejections over runs) separates them, but is estimated from few observations for the tests ranked low.
- **Joint rejections are never observed.** Validation stops at the first rejection, so the record of a candidate never shows two tests rejecting it. Whether two tests reject the same candidates, a relation that selection and redundancy arguments rely on, cannot be read from the record, and a rule that needs it cannot be fed by the session's own runs.
- **Cost estimates need repeated runs.** One rejection already shows that a test can reject; a cost estimate is an average of timings and becomes reliable only after a test has run several times. Each test's cost on the faulty program is measured once before repair, but its cost on a candidate is observed only when it runs on that candidate.
- **The rates need not be stationary.** In a conversational loop the failing test is shown to the model, which then aims at it; a test that has just been reported may reject the next candidate less often. A rate pooled over the whole session assumes otherwise.

A history-based order is therefore learned from data that its own early stopping withholds.

The cascade view. This is the observation model of *cascading bandits* [20], a learning-to-rank setting in which a user scans a ranked list and stops at the first attractive item; here the validator scans the ordered tests and stops at the first rejection. Two facts carry over. First, if outcomes are independent, whether a test is reached does not depend on its own outcome, so its rate among the runs that reached it is unbiased; with correlated outcomes the same rate is the rate conditional on reaching, the target of Section 2.3. Censoring thus costs observations more than accuracy. Second, that cost falls on the tests ranked low: a test whose rate is underestimated is rarely reached, so its estimate is rarely corrected. Reweighting the record by the inverse probability of reaching each test does not help, because a deterministic order reaches a test with probability zero or one. An order obtains the missing observations only by *exploring*: running an uncertain test earlier than its estimate alone would.

2.6 Benchmark

Repair tools are evaluated on benchmarks of real bugs, each shipped with a test suite that exposes the bug. This thesis uses **Defects4J**, the common benchmark of Java repair [12]: real bugs from open-source projects, each with the developer's fix and at least one test that fails before the fix and passes after it. UniAPR [13] and the study of test selection in repair [18] were also evaluated on it. Other benchmarks differ in what a test is: in ConDefects [21] a test is one run of a whole contest program, and in SWE-bench [22] a test of a large Python repository. Two properties of a benchmark decide how much a validation technique can help: how fine-grained a test is, which decides whether test selection has anything to remove, and how much test costs differ within one bug, which decides whether the order of tests matters in seconds.

Chapter 3

Related Works

Related work falls into three parts: the LLM-based repair systems that generate and validate candidate patches; the studies that measure what such systems cost; and the techniques that make a repair loop cheaper. The techniques are grouped into five classes according to the part of the cost they reduce. Test prioritisation, the group this thesis works on, is reviewed in depth; the others briefly.

The review is a structured literature review, not a systematic one: a keyword search of the major digital libraries and software-engineering venues for LLM-based repair and for patch-validation cost, followed by backward and forward snowballing from four seed papers [9, 13, 14, 18].

3.1 LLM-Based Repair Systems

A recent survey classifies LLM-based repair systems by how much control the model has over the repair loop [6]; they also differ in which part of the loop they try to improve. Table 3.1 groups representative systems into three families.

Table 3.1: Representative LLM-based repair systems, what they optimise, and how they use validation.

Family	Systems	Optimises	Use of validation
Conversational	ChatRepair [3]	patch quality from failure feedback, at low cost per bug	every candidate is tested; failures return to the conversation
Autonomous agents	RepairAgent [4], SWE-agent [23], AutoCodeRover [24]	localisation and context gathering through tools	the agent decides when to run tests
Agentless pipelines	Agentless [25]	a fixed localise–repair–validate pipeline with low, predictable cost	selected regression tests and generated reproduction tests filter and rank candidates

Conversational and pipeline repair. ChatRepair [3] alternates generation and validation in one conversation: earlier incorrect patches and their test failures stay in the prompt, so the model can avoid repeating them. It reported fixing 162 of 337 bugs at about \$0.42 per bug, yet 65 % of the patches it generated were duplicates of earlier ones [8]. Agentless [25] replaces free choice of actions with fixed phases

(localisation, sampling of several candidates and validation), which gives predictable cost and far fewer tokens than tool-driven agents [6].

Agent-, search- and reasoning-based repair. Autonomous agents such as RepairAgent [4], SWE-agent [23] and AutoCodeRover [24] let the model choose tools to read, edit and test code; they improve localisation and context at the price of long sessions, and RepairAgent spends 99 % of its repair time executing tools. Cost figures appear for only a few systems [6].

3.2 Validation in Current Tools

How current tools validate, deduplicate and stop was read from the code of six open-source repair agents and from the reports of deployed systems (Table 3.2). The review leads to four findings. No tool orders tests by how likely they are to fail or what they cost, and none stops at the first failing test; validation is left to the model, run as one fixed command, or run per patch after generation. Only AutoCodeRover caches validation results, and only by exact text; Agentless normalises patches, but only to vote among them. All tools set budgets in dollars, steps, rounds or timeouts, but only OpenHands detects a stuck agent, from repeated actions and errors. No tool reports why it gave up beyond an exit status, except an exit summary in Meta's repair agent [26]; SWE-agent submits its current diff anyway. Every tool tracks tokens and money; none accounts for the time spent running tests, although RepairAgent spends 99 % of its time in tools [4].

Table 3.2: How current repair tools validate, deduplicate and stop, read from their code (open-source agents, November 2026) or reports (deployed systems).

Tool	Validation	Repeats	Budget, stop, give-up
SWE-agent [23]	the model runs a reproduction script through the shell	identical parallel actions dropped	$ and call limits, timeouts; on exit submits the current diff
OpenHands [27]	left to the model	notice after repeated identical calls	step and $ limits; stuck detector; status only
Agentless [25]	regression and reproduction tests on every sample, after generation; no feedback	normalised, for voting only	number of samples, timeouts
AutoCode-Rover [24]	reproducer, an LLM reviewer, then the developer's tests	exact-text cache of results	retry and round caps; a reason string
Aider [28]	linter, then the user's test command	notice for an edit already applied	three reflections
RepairAgent [4]	the whole Defects4J suite	notice for a repeated command	40 cycles
SapFix (Meta) [29]	reproduction, analysis, CI tests	one fix published	dropped if unreviewed in 7 days
Repair agent (Meta) [26]	the broken tests, analysis, an LLM judge	failures grouped by cause	timeout and cap; exit summary
Passerine (Google) [30]	the bug-reproduction tests	—	20 trajectories of 25 steps

3.3 Cost Measurement in LLM-Based Repair

Studies that report what LLM-based repair costs fall into three kinds.

Cost per bug, as a by-product. Repair systems report a cost next to their success rate, each in its own unit: ChatRepair in API dollars per bug [3], RepairAgent in tokens and in repair time per bug [4], and CigaR, which makes token cost its objective, in tokens per bug against a conversational baseline [5]. These figures compare systems on one side of the bill and are not normalised across studies [6].

Cost-aware comparisons. A second kind of study holds the budget fixed and asks which strategy spends it better. Independent repeated prompting repairs more bugs than a conversational repairer at the same number of attempts [8]. For software agents, SWE-Effi re-scores SWE-bench systems by their resolve rate per token and per unit of time, and finds that effectiveness depends on how a scaffold pairs with its model, and that unresolved issues consume far more resources than resolved ones [10].

The cost of execution. Lin et al. [31] isolate execution in agents. Every agent examined relies on code execution, which has become standard practice, but applies it indiscriminately: for commercial agents on SWE-bench, prohibiting execution changed the resolve rate by only 1.25 percentage points, a difference that was not statistically significant, while saving 56–62 % of tokens and 48–54 % of wall-clock time for one of them.

What is not measured. All three kinds treat a repair session as a whole, or remove one of its parts entirely. None splits the cost of one loop into generation and validation, nor validation into repeated, rejected and accepted candidates (Section 2.2). Without that split, the share of a session that a validation-cost technique can reach is unknown, and so is the largest saving it could deliver.

3.4 Classification of Cost-Reduction Techniques

Table 3.3 sorts the techniques into five groups by the part of Equation (2.2) they cut. Because the groups cut different parts, they can be combined. Groups A and B are the two classical problems of regression testing: *selection* decides which tests to run, *prioritisation* the order in which they run.

3.5 Test Prioritisation

Principle. Validation can stop as soon as one test fails, because one failure proves the patch wrong. The order of the tests then decides how much is spent before that

Table 3.3: The five groups of cost reduction. "Learns" marks groups that use what the session has observed.

Group	Cuts	Safe	Language-bound	Learns	Representative work
A test selection	tests that do not reach the change	if sound	yes	no	RTS in APR [18]
B test prioritisation	$V_{\text{rej}} - V^*_{\text{rej}}$	yes	no	yes	FRTP [9], PIT [11]
C execution acceleration	start-up share	yes	yes	no	UniAPR [13], ExpressAPR [14]
D patch reduction	V_{rep} and equivalent candidates	mostly	partly	partly	verdict cache, test-equivalence [32]
E generation-side reduction	G_{rep} and rounds	—	no	yes	ChatRepair [3], CigaR [5]

failure: a failing test placed first costs one run, the same test placed last costs the whole suite. Ordering removes no test, so for deterministic tests it never changes a verdict.

Key techniques. The information an order is built from (Table 3.4).

Table 3.4: Test prioritisation, by the information it ranks on.

Information	Rule	Representative work
Coverage	tests covering more (new) code first	TP_{tot}, TP_{add} [17]
Failure history	tests that rejected more candidates first	FRTP [9]
Cost	likelihood of failing per unit of cost	least-cost testing [15], APFD_C [19]
Location tiers	failing tests, then tests in the package of the patched code, then the rest	ExpressAPR [14]
Coverage, cost and name	tests covering the change only, fastest first, the unit test of the changed class favoured	PIT [11]
History across variants	tests that killed other mutants first	FaMT [33]
History across CI cycles	a learned policy, rewarded for early failures	RETECS [34], COLEMAN [35]

Coverage-based ordering comes from regression testing (Section 2.4) and reads only the program. FRTP is built for repair: it runs first the tests that rejected earlier candidates in the same session, information that validation provides at no extra cost, and it matched GenProg's success rate while validating faster [9]. Cost entered prioritisation in regression testing [19], where test costs are assumed known in advance; none of the repair orderings uses it, although the run of the faulty program before repair already measures each test's cost. ExpressAPR orders tests by location in fixed tiers: the failing tests, then the tests in the package of the patched code, then the rest [14]. It uses neither test costs nor the session's history.

Mutation testing faces the same problem as repair: many variants of one program, each to be rejected by any failing test, a duality ExpressAPR builds on [14]. PIT,

the most used mutation tool for Java, runs only the tests that cover the mutated line, fastest first, favours the unit test of the mutated class, and stops at the first kill [11]. FaMT orders the tests for each mutant by how they fared on other mutants [33]. Together they use the signals a repair ordering could use, namely cost, the location of the change and history across variants, but in a setting where the variants are known in advance, the change is located exactly by an instrumented coverage run, and no failure is reported to anyone. In LLM-based repair the candidates arrive one at a time and are few, and the failing test that validation reports becomes the model's next input.

Learned orders exist in continuous integration, where the order is learned across commits: RETECS uses reinforcement learning [34] and COLEMAN a multi-armed bandit [35], both rewarded for detecting failures early. Ranking under stop-at-first-success has its own bandit model, cascading bandits, with UCB and Thompson sampling algorithms [20, 36] and a variant in which examining each item has a cost [37]. These methods learn over hundreds or thousands of cycles. A repair session validates a handful of candidates, so whatever a learned order explores must pay off within a few rounds.

Limitations. FRTP ranks on a *count* of past rejections [9], which cannot tell a test never run from one run often that never rejected, and no repair ordering reads how long a test takes; mutation tools do, but statically, from an instrumented run, and with rules that join the signals in sequence rather than in one estimate. History-based orders also learn from censored evidence: stopping at the first failure records only one failing test per rejected patch (Section 2.5). Ordering saves anything only when validation stops at the first failure, and after test selection there are fewer tests to order, so the best order can change [18]. Finally, the unit of measurement matters: prioritisation studies count test executions, which reproduce across machines [18], but when test costs differ an order that runs more cheap tests can finish sooner than one that runs fewer expensive ones, so counts and seconds can rank two orders in opposite directions.

3.6 Other Cost-Reduction Techniques

A: Test selection. A test that never executes the code a patch modifies gives the same result on the patched program as on the original, so it can be skipped. Lou et al. [18] measured regression test selection inside twelve repair systems on more than two million patches: every granularity (class [38], method or statement) substantially improved efficiency, finer granularity was better, and combining selection with prioritisation was best. Sound selection needs a dependency tracker for the language, and it has nothing to remove when every test runs the whole program.

C: Execution acceleration. Part of a test run is overhead: recompiling the patched program and starting the runtime. UniAPR keeps one JVM alive and reloads only the patched classes, speeding repair up by over an order of magnitude [13]; ExpressAPR

borrows five mutation-testing accelerations, including compiling many candidates into one program, and validates 137× faster than plain validation [14]. Both are built for the JVM; both also prioritise tests, ExpressAPR with the static location tiers of Table 3.4.

D: Patch reduction. Some candidates need not be run at all. The simplest reduction is a verdict cache: a candidate textually identical to one already validated is given the stored verdict, which matters for LLMs, since 65 % of the patches ChatRepair generated were duplicates [8]. Test-equivalence analysis runs one candidate per group of candidates that behave alike on the tests [32]. Equivalence analysis needs a program analysis for the language.

E: Generation-side reduction. A loop that needs fewer rounds, or wastes fewer on candidates it has already seen, is cheaper on both sides. The main levers are what the model is shown after a failure, such as the failing test in a conversation [3], and how the budget is spent: many candidates per call to amortise the prompt [5], or independent re-sampling instead of conversational refinement [8]. These techniques are evaluated by success and tokens; the validation they cause or save is not reported.

3.7 Research Gap

Three observations close the review. First, every validation-side group has a technique with a measured saving: test selection [18], test prioritisation [9], execution acceleration [14] and equivalence-based patch reduction [32]. However, all were measured with pre-LLM repair systems, on Java or C programs.

Second, none of the LLM-based systems of Section 3.1 reports using test prioritisation, and where these systems report cost, they report one side of the bill in a unit of their own (Section 3.3). How the cost of an LLM-based loop splits between generation and validation, and within validation, is not known.

Third, of the validation-side groups, test prioritisation transfers most directly (Table 3.3): it is safe for deterministic tests, language-independent, and learns from the loop, while execution acceleration is bound to one runtime, test selection to a sound dependency tracker, and patch reduction beyond a verdict cache to program analysis. But it acts only on the cost of rejecting invalid patches, so its value depends on how large that cost is in the whole session, a quantity that no study has measured.

Open problems. Together, the observations give five open problems, which the proposed system addresses (Chapter 4).

O1 The cost composition of a repair loop is unmeasured. Without it, the upper bound on what a technique can save in a given setting is unknown.

O2 The repair orderings were built for another kind of loop. FRTP was designed with search-based generators, which validate many candidates [9]; an LLM-based loop validates few, so a session-local history is short, and in a conversational loop the failing test is also the model's next input.

O3 Count instead of rate, under censored evidence. A count of past rejections cannot tell a test never run from one that never rejects, and early stopping hides every test after the first failure.

O4 Cost and the location of the patch are used only statically. The loop times every test it runs and knows which class each candidate modifies; no repair ordering uses the costs, and the location enters repair orderings only as a fixed tier (ExpressAPR). Mutation tools combine cost and location, but statically and from an instrumented run (PIT); none estimates them online from the session's own runs.

O5 Repeats and stuck sessions are not managed. Most of the patches a conversational repairer generated were repeats of earlier ones [8]; they are generated, and often validated, again, since only one tool caches results, by exact text. Sessions stop at a fixed budget; only one agent framework detects that it is stuck, and none reports why it gave up (Table 3.2).

Chapter 4

Proposed System

This chapter presents the proposed system, a *validation layer* placed between an LLM repair loop and the project's test runner. It has three modules, one per waste of Figure 1.1. Section 4.1 states what a practical repair bot needs; Section 4.2 shows the layer; the next three sections describe the modules.

4.1 Usage Scenario and Requirements

Scenario. A test fails in continuous integration. A repair bot is given the faulty code and tries to fix it within a budget set by the team. It returns either a plausible patch for a developer to review, or a report of why it gave up. Developers name CI as the place they most want repair to run [7], and deployed systems work this way: they act on failures found in CI and send every fix to a reviewer [26, 29, 39]. Table 4.1 lists what such a bot needs.

Table 4.1: What a repair bot in CI needs, and the evidence.

Requirement	Evidence	Served by
P1 bounded time	63 % of 103 practitioners would not wait over an hour [7], though correctness matters more than speed [40]	M1 Repeat Guard, M2 Test Prioritiser
P2 predictable cost	budgets decided the adoption of repair at Bloomberg [39]	M3 Budget Guard
P3 drop-in	developers want repair inside the CI pipeline [7], fitting their workflow [41]	the layer
P4 trustworthy	a fix must pass all CI checks and be reviewed [39]	M1 Repeat Guard, M2 Test Prioritiser
P5 explain a failure	unexplained fixes are ignored [41]; developers value the fault location and tests [7]	M3 Budget Guard

The evidence is strong for P3 and P4 and mixed for P1, so the layer aims at a bounded time, not the shortest. P2 and P5 are supported only indirectly; they are design choices the evidence favours.

4.2 System Overview

Figure 4.1 shows one repair session. In each round the model proposes a candidate. The **Repeat Guard** (M1) checks whether it has seen the candidate before; if so, it reuses

the stored result. Otherwise the **Test Prioritiser** (M2) orders the tests with ARC, and the test runner runs them until the first failure. After a rejection, the **Budget Guard** (M3) checks that the session is still making progress and has budget left. If so, the next round starts, and the model is shown either the failure or a notice that the candidate was a repeat. Otherwise, the session ends with a give-up report.

The repair loop interacts with the layer through a single call, which takes a candidate and returns the verdict, the feedback for the model and whether the session should continue. The layer therefore also determines what the model is told and when the session stops:

```
result = layer.validate(candidate)
result.verdict    # plausible, rejected, does not compile, or repeat
result.feedback   # text to show the model in the next round
result.go_on      # false when the Budget Guard stops the session
result.report     # the give-up report, when go_on is false
```

The layer is configured with a budget per bug and needs no instrumentation of the project.

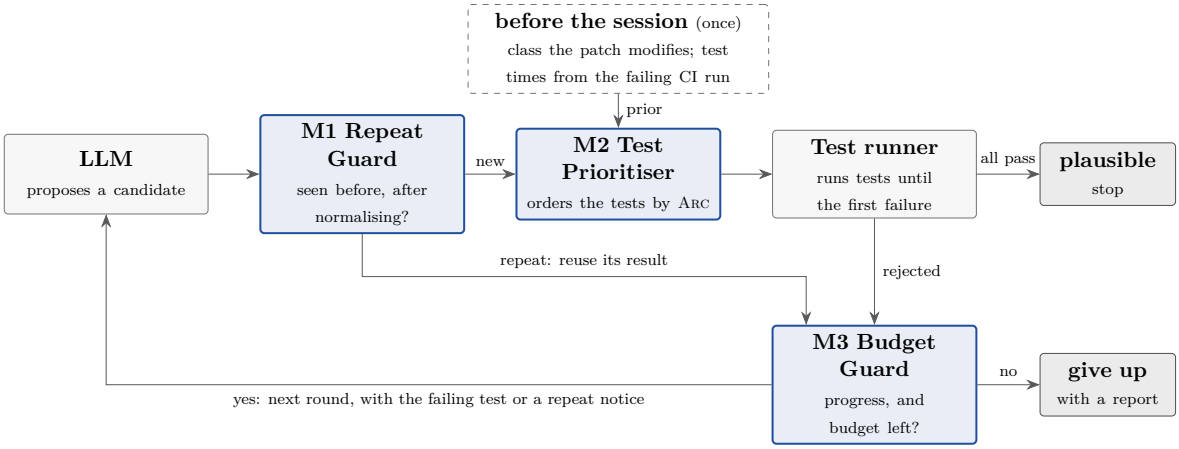


Figure 4.1: One repair session with the validation layer. The three modules (highlighted) sit between the repair loop and the project's test runner.

Table 4.2: The three modules: what each improves, the cost it targets, and how.

Module	Improves	Targets	How
M1 Repeat Guard	no time or money spent twice on the same patch	V_{rep}	normalised cache; repeat notice
M2 Test Prioritiser	wrong patches rejected sooner	V_{rej}	order by ARC, learned during the session
M3 Budget Guard	predictable cost; a reason when it fails	failed sessions	stop when stuck or out of budget; give-up report

A session, round by round. Table 4.3 follows one bug through the three modules, with the Budget Guard set to stop after $r = 2$ rounds without progress.

Table 4.3: One session of a bug in `DateUtils`, round by round (illustrative).

Round	Candidate	Repeat Guard and Test Prioritiser	Budget Guard
1	new patch P_1	ARC runs `DateUtilsTest` early (named after the modified class); it fails with error A	progress (new failure)
2	P_1 with a variable renamed	repeat: result reused, no test run; the model is told	no progress (1)
3	new patch P_3	`DateUtilsTest` passes; `CalendarTest` fails with error B	progress (new failure)
4	new patch P_4	ARC now runs `CalendarTest` first; it fails with error B again	no progress (1)
5	new patch P_5	fails with error B again	no progress (2): stop

Give-up report: stopped for lack of progress; 5 rounds, 4 distinct patches, 1 repeat; error B three times, error A twic

4.3 Module M1: Repeat Guard

Principle. A patch that the model has already proposed in the session does not need to be validated again.

Normalisation. The Repeat Guard (M1) keeps a cache of the session's candidates. Each candidate is first *normalised*: it is parsed into a syntax tree, comments are removed, local variables are renamed in order of appearance $(v_0, v_1, \dots)$, and the tree is printed back. A candidate whose normalised form is already in the cache is a *repeat*; it gets the stored result without running any test. The Repeat Guard also tells the model, in the next prompt, that the patch was already tried and which test rejected it.

Safety. Normalisation only removes differences that cannot affect behaviour, namely layout, comments and local variable names. Semantically equivalent patches (for example $a+b$ and $b+a$) are not merged. Two patches therefore share a result only when they behave identically, so reusing the result is safe for deterministic tests.

4.4 Module M2: Test Prioritiser

Principle. A wrong patch is rejected most cheaply when the test classes run in decreasing q_t/c_t: the chance q_t that class t rejects it, divided by its running time c_t (Proposition 2.2). Neither quantity is known in advance, so ARC (Adaptive Rate-over-Cost) estimates both during the session and runs the classes in decreasing

$$\text{score}_t \;=\; \frac{\theta_t}{\hat{c}_t},$$

where θ_t is a draw of the failure chance and $\hat{c}_t$ the estimated running time. The paragraphs below define the two parts; Algorithm 1 puts them together.

Failure chance: the prior. A wrong patch usually fails only a few of many test classes, most often the classes that test the code it changes. Before any run, each class therefore gets a prior π_t from its location. The classes fall into three groups: those named after the modified class (their simple name starts with its name, e.g. `StringUtilsTest` for `StringUtils`), the other classes of its package, and the rest. Each group g receives a share M_g, split equally among its $|T_g|$ classes:

$$\pi_t = \frac{M_g}{|T_g|} \qquad \text{for each class } t \text{ in group } g.$$

The shares sum to one, so the priors do too, which corresponds to about one failing class per wrong patch; the share of an empty group is spread over the others. The provisional shares are 0.60 (named after the modified class), 0.25 (same package) and 0.15 (the rest); the development set fixes them as the share of the failing classes of wrong patches that falls in each group (Section 5.10).

Failure chance: what the session observes. The prior counts as one imaginary run of the class, which failed with weight π_t and passed with weight $1 - \pi_t$; each real run adds one to the failures or to the passes. After n_t runs of which k_t failed, the two counts and the estimated failure chance are

$$\alpha_t = \pi_t + k_t, \qquad \beta_t = (1 - \pi_t) + (n_t - k_t), \qquad \hat{q}_t = \frac{\alpha_t}{\alpha_t + \beta_t} = \frac{\pi_t + k_t}{1 + n_t}.$$

The estimate $\hat{q}_t$ rises when the class fails and falls when it passes, and the observed runs soon outweigh the prior. In Bayesian terms, $\hat{q}_t$ is the mean of the posterior $\mathrm{Beta}(\alpha_t, \beta_t)$ that the prior $\mathrm{Beta}(\pi_t, 1 - \pi_t)$ gives after these runs.

Failure chance: exploration. Ranking by $\hat{q}_t$ alone has a blind spot. Early stopping observes a class only when the order reaches it (Section 2.5), so a class whose $\hat{q}_t$ is too low is rarely run, and its estimate is rarely corrected. ARC therefore ranks by a random draw from the posterior instead of its mean,

$$\theta_t \sim \mathrm{Beta}(\alpha_t, \beta_t),$$

made anew for each candidate: Thompson sampling [42], the rule that does best in practice for cascading bandits, the learning model of stop-at-first-success ranking [36]. A class with many runs has a narrow posterior and draws close to $\hat{q}_t$; a class with few runs has a wide one, and sometimes draws high enough to move up and be observed. The exploration therefore needs no parameter of its own and fades as the session observes more.

How much it explores depends on the prior being calibrated. With the provisional shares, in a suite of 60 classes (one named after the modified class, five in its package) and before any run, the named class is drawn first for about three candidates in four,

and a class of the third group for fewer than one in ten. The draws are seeded by the session and the candidate, so every run can be reproduced. Ranking by $\hat{q}_t$ instead of θ_t is a variant in RQ4, which tests whether exploration pays within the few candidates of a session.

Running time, $\hat{c}_t$. The bot is started by a failing CI run, which already timed every class on the faulty program; call this time c_t^{CI}. ARC starts from it and switches to the average of the session's own runs once the class has run:

$$\hat{c}_t = \begin{cases} s_t/n_t & \text{if } n_t > 0, \\ c_t^{\mathrm{CI}} & \text{otherwise,} \end{cases}$$

where s_t is the total time of its n_t runs. A class with no recorded time, such as a new test, is estimated from the size of its test file, at the rate observed so far.

Order. For each new candidate, ARC draws θ_t for every class, runs the classes in decreasing $\theta_t/\hat{c}_t$, stops at the first failure, and updates the counts of the classes that ran (Algorithm 1). It needs no extra test run, no model call and no instrumentation.

Algorithm 1 ARC for one new candidate

Require: candidate p; test classes T; for each class t: counts α_t, β_t (initially π_t and $1-\pi_t$), runs n_t and total time s_t (initially 0)

for each $t \in T$ **do**
  draw $\theta_t \sim \mathrm{Beta}(\alpha_t, \beta_t)$
  $\hat{c}_t \leftarrow s_t/n_t$ if $n_t > 0$, else c_t^{CI}
end for
for t in T sorted by $\theta_t/\hat{c}_t$, highest first **do**
  run t on p, taking time c
  $n_t \leftarrow n_t + 1$; $s_t \leftarrow s_t + c$
  if t failed **then**
    $\alpha_t \leftarrow \alpha_t + 1$; **return** "rejected by t"
  else
    $\beta_t \leftarrow \beta_t + 1$
  end if
end for
return "plausible"

Example. Table 4.4 compares ARC with FRTP, which ranks tests by how many patches they have rejected. FRTP ties T_1 with T_2 and T_3 with T_4. ARC separates them: T_1 failed 8 of 10 times and is fast, so it goes first; T_4 has never run but is named after the modified class, so it goes second; T_2 is thirty times slower than T_1; and T_3, though fastest, has passed 25 times and goes last. This is the order of the means $\hat{q}_t$. With sampling, T_4, which has never run, comes first for about a quarter of the candidates, while T_3 stays last for almost all.

Table 4.4: FRTP and ARC on four of 100 test classes (illustrative values).

Test class	failed k_t	runs n_t	time $\hat{c}_t$ (s)	$\hat{q}_t$	$\hat{q}_t/\hat{c}_t$
T_1	8	10	0.04	0.73	18.2
T_2	8	30	1.20	0.26	0.22
T_3	0	25	0.03	0.00006	0.002
T_4 (named after the modified class)	0	0	0.05	0.60	12.0
FRTP: $\{T_1, T_2\}$ tied, then $\{T_3, T_4\}$ tied;			ARC: T_1, T_4, T_2, T_3		

Relation to existing work. The information used by ARC is not new. The mutation tool PIT already orders tests by cost and by the location of the change [11], and FaMT uses history across mutants [33]. The difference is that ARC combines these signals in a single estimate, updates it during the session without instrumentation, and is designed for patches that arrive one at a time and whose reported failure is shown to the model (Section 3.5). In bandit terms, ARC is a cost-aware cascading bandit [20, 37] solved by Thompson sampling, with two features a repair session needs: a prior from the location of the change, because the session is too short to learn from scratch, and test times from the failing CI run, so cost is known before the first candidate.

4.5 Module M3: Budget Guard

Principle. The Budget Guard ends sessions that make no progress, keeps the cost per bug within a budget and reports why a session stopped. Failed sessions account for most of the cost [10], and current tools stop only at a fixed limit without giving a reason (Section 3.2).

Progress. A rejected candidate leaves a *failure signature*: the test that rejected it, the exception type and its message, without line numbers, which change when a patch is reformatted. A round makes progress when its candidate is new and fails in a new way. A repeat, or new code that fails exactly as before, makes no progress.

When to stop. The Budget Guard (M3) counts the rounds since the last progress and stops when the count reaches r. It also stops before a round that the remaining budget B (in time, tokens or money) cannot pay for, judged by the average cost of the rounds so far. The round limit $R_{\max}$ stays as a hard cap. The values of r and B are chosen on the development set (Section 5.10).

Give-up report. A session that ends without a plausible patch returns: why it stopped; how many rounds, distinct candidates and repeats it had; what it spent; and its failure signatures, most frequent first, with the test, the error and an example patch. The signatures show where the model repeatedly failed, which is the kind of

information (fault location and tests) that developers find most useful when they fix a bug themselves [7]. Figure 4.2 shows an example.

```
Give-up report  (bug Lang-65, session 2)
Stopped:  no progress for 3 rounds
Rounds:   9   distinct patches: 7   repeats: 2
Spent:    410 s  (model 120 s, validation 290 s), 38k tokens
Failures, most frequent first:
  4x  DateUtilsTest.testTruncateLang59
      AssertionFailedError: expected <...MDT 2004> but was <...MST 2004>
  2x  DateUtilsTest.testRound
      ArithmeticException: Calendar value too large
  1x  compile error: cannot find symbol 'millis'
Example patch for the most frequent failure: round 7
```

Figure 4.2: An example give-up report (illustrative values).

4.6 When the Layer Helps

- **Module M1 (Repeat Guard)** saves in proportion to how often the model repeats itself.
- **Module M2 (Test Prioritiser)** saves most when test times differ a lot and a session has many wrong patches; it cannot reduce the cost of the final, accepted patch, which runs every test in any order.
- **Module M3 (Budget Guard)** saves most when sessions go on long after the model is stuck.
- A slow model, compilation and JVM start-up are untouched, so they dilute the saving end to end; RQ1 measures by how much.

Chapter 5

Evaluation Plan

The evaluation has two levels (Table 5.1). **E1** tests the test orderings in a controlled simulation: the patches are made by mutation, so every ordering sees exactly the same patches. **E2** and **E3** run the repair loop live, with a real model and real feedback.

Table 5.1: The three experiments.

Experiment	RQ	What is run
E1 orderings on mutants	RQ2, RQ4	five orderings and their variants on simulated sessions of mutated patches; 100 bugs; no model call
E2 orderings, live	RQ1, RQ2	the repair loop with each of four orderings and a real model; 100 bugs, 3 independent sessions per bug, each up to 20 rounds
E3 whole layer, live	RQ3	the full validation layer against the plain loop; 50 bugs, 3 sessions per bug, each up to 20 rounds

5.1 Bugs and Setup

Bugs. Defects4J [12] version 3.0.1 has 854 real Java bugs from 17 projects. A bug is used if (i) it can be reproduced, that is, its failing tests fail before the fix and pass after it; (ii) the fix changes one source file; and (iii) its test suite has at least ten test classes and runs in under 5 minutes, which keeps the audits within budget (Section 5.9). From these, 120 bugs are drawn at random, spread evenly over the projects: 20 form a *development set*, used to check the pipeline and fix every setting in advance, and 100 form the *evaluation set*, on which nothing is tuned. E3 uses 50 of them.

Faulty code given. Finding the faulty code is not studied. As is common in LLM-based repair [3], the model is given the methods that the developer's fix changes. Every candidate patch replaces these methods and nothing else. This favours repair success [43] but is the same for every configuration.

Unit of ordering. Tests are ordered by test class. Validation compiles the candidate, runs the classes in the chosen order, and stops at the first failing class. A candidate is plausible when it passes the whole suite.

5.2 What Is Compared

Five orderings are compared (Table 5.2), in two settings. *Without a trigger step*, no ordering puts the originally failing tests first, so the orderings are compared on the same information. *With a trigger step*, every ordering first runs the *triggering classes* (the test classes that contain a test failed by the faulty program) and then orders the rest. Since practical tools already run failing tests first, the second setting examines whether the order of the remaining tests still matters. The trigger step is a prefix that can be combined with any ordering, and both settings are computed from the same audits.

Table 5.2: The orderings compared.

Ordering	Rule	Role
Default	Defects4J's own order	reference
FRTP [9]	classes that rejected more patches in the session first	baseline
Coverage (PIT rule) [11]	only classes that run the faulty methods; fastest first; the class named after the modified class first	baseline
ARC	Algorithm 1	proposed
Oracle	the cheapest failing class, known in hindsight	lower bound

Coverage follows the rule of PIT, the mutation tool whose problem is closest to ours, and is the strongest existing baseline. It needs one instrumented run of the faulty program before the session, to know which classes run the faulty methods; classes that do not can safely be skipped [17], so Coverage is the only ordering that also shortens the run of the accepted patch.

Variants of Arc (RQ4). Each changes one part of ARC: no cost term; cost only (fastest first); a flat prior; no location prior, which also shows ARC without the indirect hint that a class named after the modified class gives; a wrong location; no exploration (ranking by the mean $\hat{q}_t$ instead of a draw); start times estimated from file size instead of the CI run; FRTP with a cost tie-break; and ARC on the classes Coverage selects.

5.3 How Cost Is Measured

Audit. Every distinct candidate is compiled once and run on *every* test class, recording each class's result and time. From this record, the cost of any ordering on that candidate is computed: compilation, JVM start-up, and the classes the ordering would run up to its first failure. All orderings are thus charged from the same measurement. Table 5.3 shows the computation for one candidate.

The last line shows why the trigger-step setting matters: a candidate that fails a triggering class costs the same under every ordering, so the orderings differ only on candidates that pass the triggering classes.

Table 5.3: Charging orderings from one audit (illustrative). The candidate fails classes C and D; compilation and JVM start-up are the same for every ordering and are left out.

	A	B	C	D	E
Time (s)	0.2	3.0	0.1	1.0	0.5
Result	pass	pass	fail	fail	pass

Default (A, B, C, stop): $0.2 + 3.0 + 0.1 = 3.3$ s
FRTP, if D rejected earlier patches (D, stop): 1.0 s
ARC, if it ranks E before C (E, C, stop): $0.5 + 0.1 = 0.6$ s
Oracle (cheapest failing class C): 0.1 s
With a trigger step, if C is a triggering class: 0.1 s for every ordering

How time is measured. The classes of a candidate run one after another in a single JVM, as build tools run a test suite. A JUnit 4 runner, started once per candidate, records for each class its result, its wall-clock time and the CPU time of the whole JVM process while the class runs (all threads, including JIT compilation and garbage collection). JVM start-up is charged once per candidate, as the process time spent outside the classes, and compilation as the CPU time of the build. A JVM per class would add a start-up to every class and penalise the orderings that run more classes; a shared JVM charges it once, as in practice. A class that runs longer than 60 seconds counts as failed; if the JVM dies inside a class, that class counts as failed and the remaining classes run in a new JVM. The main unit is CPU seconds, which depends less on the load of the machine than wall-clock time; wall-clock time is recorded for every class and reported as well.

What is recorded. For each model call: latency and tokens. For each audit: compile time, JVM start-up, and the result, wall-clock time and CPU time of each test class. Costs are reported in time, CPU time, tokens and money, using one fixed price list: the model's list price and $0.05 per CPU-hour of validation.

Reliability checks. Each test class runs three times on the fixed program; classes that change outcome are excluded as flaky. A class is stopped at a time limit, and a patch that hits it is rejected. On a random tenth of the candidates, the audit is repeated in two other orders to check that results do not depend on test order.

5.4 E1: Orderings on Mutants

Why mutants. In a loop with feedback, the test order determines which failure the model sees and therefore influences the next patch, so two orderings cannot be compared on the same sequence of patches. Mutants do not depend on feedback, so every ordering can be evaluated on the same patches in the same order. Mutants are a standard stand-in for real faults when evaluating test ordering [44], and in mutation-based repair tools the candidate patches are mutants [45]. Unlike a mutation testing

study, E1 measures the cost of validating a repair session rather than the number of mutants a suite kills.

Patches. Mutants are generated with Major [46], the mutation tool shipped with Defects4J, in four groups (Table 5.4). The main groups lie in the faulty methods, where the model's patches lie; two extra groups lie elsewhere, to check that results do not depend on this. For each bug, mutants are drawn at random until 5 per group compile and fail at least one test, auditing at most 10 per group: about 20 wrong patches per bug, 2,000 in all.

Table 5.4: Groups of mutated patches.

Group	Mutated code	Stands for	Use
S1	faulty methods, fixed program	a fix that breaks something else	main
S2	faulty methods, faulty program	an attempt that leaves the bug	main
S3	another method of the faulty class	a change in the wrong method	check
S4	another class used by the failing tests	a change in the wrong class	check

Sessions. A simulated session is a random order of a bug's wrong patches, ending with the developer's fix, which passes every test. Main sessions use the 10 patches of S1 and S2; check sessions use all 20. Each bug gets 10 random orders, and every ordering runs on the same ones, in both settings. The main measure is the validation cost of Arc relative to each baseline, per bug.

5.5 E2 and E3: The Loop, Live

The loop. A conversational loop in the style of ChatRepair [3]. The first prompt gives the faulty methods, a failing test and its error message. After each round the outcome is fed back (Table 5.5); Figure 5.1 shows a first prompt and a feedback message. A session ends at the first plausible patch or after 20 rounds.

Table 5.5: Feedback after each round.

Outcome	Feedback to the model
A test fails	the first failing test, its line and error; ask for a different fix
Does not compile	the compiler error; ask for a corrected version
Unreadable answer	the answer could not be read as the given methods
Repeat (E3 only)	the patch was already tried and which test rejected it

Model. One model through an API: OpenAI's `gpt-6-luna`, the smallest tier of the GPT-6 family, at temperature 0.8, so that the three sessions of a bug differ, and at most 2,048 output tokens per answer. The dated snapshot in use when the runs start is

First prompt

```
The following Java method contains a bug:

    private static void modify(Calendar val, int field, boolean round) {
        ...
    }

It makes this test fail:
    Test:  DateUtilsTest.testTruncateLang59
    Error: AssertionFailedError: Truncate Calendar.SECOND
           expected:<Sun Oct 31 01:02:03 MDT 2004> but was:<... MST 2004>

Return the complete fixed method in one Java code block.
```

Feedback after a rejected patch

```
Your fix is still incorrect. It fails this test:
    Test:  DateUtilsTest.testRoundLang346
    Error: AssertionFailedError: Round Calendar.MINUTE
           expected:<10:31> but was:<10:30>
Please provide a different fix.
```

Figure 5.1: The first prompt and a feedback message for Defects4J bug Lang-65.

recorded and kept for every experiment. At its list price of $0.10 and $0.50 per million input and output tokens, the upper bound of 30,000 calls (Section 5.10) costs about $45 even if every call carried 10,000 input and 1,000 output tokens. A *session* is one repair attempt on one bug: it runs up to 20 rounds, one model call per round, and stops earlier if a plausible patch is found. Each bug gets three independent sessions per configuration. Defects4J is likely memorised by current models [47]; if most sessions ended in one or two rounds, there would be little cost to save. The model is therefore chosen on the development set by a rule fixed in advance: the median session must need at least three rounds; otherwise a weaker model is used. A small tier is the natural first choice for a repair bot that runs on every failing build. The rule fixes a setting in which sessions take several rounds; how the cost breakdown changes with a stronger model is left to future work (Section 7.4).

Fair comparison. Configurations of the same bug and session share two caches. Model answers are cached by their full prompt, so configurations get the same patches until their conversations first differ. Audits are cached by normalised patch, so each patch is tested once and every configuration is charged from the same measurement.

E2: orderings, live. The loop runs on the 100 evaluation bugs, three sessions each, with four orderings: Default, FRTP, Coverage and ARC. All use the result cache, with no repeat notice and no stopping rule, so they differ only in the ordering. E2 measures the validation cost of ARC against each baseline, and whether any ordering lowers the repair rate, since the failure it reports may be less useful to the model. The cost and headroom are also reported separately for candidates that pass the triggering classes,

because these are the only candidates whose cost still depends on the ordering once a trigger step is used. The Default sessions also answer RQ1: where the cost goes. The repeat cache and the stopping rule are evaluated on these sessions by replay, which is exact because neither changes the prompt.

E3: whole layer, live. The full layer runs on 50 bugs, three sessions each: ARC with the repeat notice on, while the Budget Guard only logs when it would stop. Its stopping is then replayed, as in E2. This separates the two effects. Comparing E3 with the ARC sessions of E2 gives the effect of the notice, and the replay gives the effect of the Budget Guard. The whole layer is compared with the Default sessions of E2. The main measure is the cost per repaired bug; the repair rate must not fall by more than a margin fixed in advance.

5.6 Metrics

Table 5.6 lists the metrics. Costs are summed over sessions before dividing, and every comparison is made per bug.

Table 5.6: Metrics.

Metric	Definition	RQ
Cost per repaired bug	total cost of all sessions / number of bugs repaired	1, 3
Cost breakdown	share of generation, V_{rep}, V_{rej} and V_{acc} in the total	1
Ordering headroom	what the best possible order would save, as a share of the total	1
Repeat rate	share of rounds whose patch is a repeat	1, 2
Validation cost ratio	validation cost of ARC / that of another ordering, per bug	2
Stopping trade-off	cost saved by the Budget Guard against plausible patches lost	2
Repair rate difference	share of sessions with a plausible patch, against Default	2, 3
Time to a plausible patch	median time from start to the plausible patch	3
Cost of failed sessions	share of the total spent on sessions without a plausible patch	1, 3
Budget overrun	share of sessions that exceed the budget	3

5.7 Statistics

The bug is the unit of analysis; the sessions of a bug are averaged first. The two main comparisons, ARC against FRTP and against Coverage, are tested in E1 and in E2 with a paired Wilcoxon test on the per-bug log-ratio of costs, with a Holm correction. With 100 bugs, a saving of 5 % is detected with power 0.8 if the standard deviation of the log-ratio is at most 0.15; live runs are noisier, and if their spread is larger the smallest detectable saving is reported. In E3, the repair rate is tested for non-inferiority; with 50 bugs a margin of 10 percentage points is reachable if the standard deviation of the paired difference is at most 0.24. If the development set shows more spread than assumed, the number of sessions per bug is raised before the number of bugs. All estimates are reported with bootstrap intervals.

The assumed standard deviations and the margin are provisional values, checked on the development set (Table 5.8).

5.8 Pilot Study

To be added. Results of a pilot on Defects4J bugs (orderings on mutated patches), about three pages. They inform the setup and are not results of the study.

5.9 Compute Budget

Most of the compute goes to audits: each runs a whole suite once. Table 5.7 gives the counts and caps; the time per audit is measured on the development set before registration. If the estimate exceeds the budget, the number of mutants per group is lowered from 5 to 3 and the suite limit from 5 to 3 minutes, in that order.

Table 5.7: Audit counts and caps.

Work	Audits
Flakiness check	3 suite runs per bug
E1 mutants	at most 40 per bug (10 per group); 4,000 at most
E2 and E3 candidates	one per distinct patch, shared by all configurations
Order check	a random tenth of all audits, run twice more
Cap per bug	2 CPU-hours in E1 and 4 in E2–E3; a bug that reaches its cap is reported and dropped

5.10 Pre-Registration

This proposal fixes the bugs, settings, metrics and analyses in advance. Table 5.8 gives the values that still depend on data, with a provisional value and the rule that fixes it on the development set; they are frozen before E1 starts on the evaluation set, and any later change will be reported.

The experiments run in order: E1, then E2, then E3. The budget is 60 repair sessions on the development set, 1,200 in E2 (100 bugs $\times$ 3 sessions $\times$ 4 orderings) and 150 in E3. A session makes at most 20 model calls, one per round, and stops early at a plausible patch; since the orderings of a bug share cached answers until their conversations differ, the 30,000 calls are an upper bound, not an expected count. Each distinct patch is audited once.

Table 5.8: Provisional values and how the development set fixes them.

Value	Provisional	Rule on the development set
Model	`gpt-6-luna`, temperature 0.8, at most 2,048 output tokens	kept if the median Default session needs at least 3 rounds, otherwise a weaker model
Prior shares of Arc	named after the modified class 0.60, same package 0.25, other 0.15	the share of the failing classes of wrong patches that falls in each group, from the development audits
Stopping count r	3 rounds without progress	the smallest r whose replay loses at most 5 % of the plausible sessions
Budget B	900 s of session time (wall-clock)	the 90th percentile of the time to a plausible patch in the Default sessions
SD of the per-bug log-ratio	0.15	measured; if larger, the smallest detectable saving is reported
SD of the repair-rate difference	0.24	measured; if larger, the sessions per bug are raised, not the margin
Non-inferiority margin	10 percentage points	not changed
Bugs	120 accepted: 20 development, 100 evaluation (50 also in E3)	the share of Defects4J bugs that pass the rules is reported from the selection log

Chapter 6

Expected Outcomes

Each research question can come out in more than one way. Table 6.1 fixes in advance how each outcome will be interpreted, so that negative results can be read as clearly as positive ones. Each module is measured on its own part of the cost, so a module that saves little does not undermine the others, and the cost breakdown of RQ1 is a result in any case.

Table 6.1: Possible outcomes and how they will be read.

RQ	If ...	then ...
RQ1	validation is a large share of the cost	the Repeat Guard (M1) and the Test Prioritiser (M2) matter; the headroom says how much they can save.
	generation dominates	the repeat notice and the Budget Guard are the main levers.
	little cost falls after the model is stuck	the Budget Guard is not needed in this setting.
RQ2	Arc is cheaper than Frtp and Coverage	learning failure probability and cost during the session is effective.
	Arc only matches Coverage	its remaining advantage is needing no instrumentation; Coverage is preferred where coverage is available.
	Arc saves on mutants (E1) but not live (E2)	the feedback it reports changes the model's patches; this is reported as a finding.
	an ordering lowers the repair rate	the failure it shows helps the model less; the saving is weighed against the loss.
	with a trigger step, ordering the rest saves little	the practical value of the Test Prioritiser (M2) is small; the cost breakdown and the mechanism of E1 still stand.
	the Budget Guard saves much at small loss	stopping unproductive sessions reduces cost at little loss.
RQ3	the layer costs less with the same repair rate	the layer meets the need; the breakdown shows which module saved what.
	the layer costs less but repairs fewer bugs	the trade-off is reported, and E2 shows which module caused it.
	no saving end to end	the savings are diluted by costs the layer does not touch; RQ1 shows which.
RQ4	one part of Arc explains most of its gain	that part is the recommendation; the rest is not needed.
	the gain disappears with a wrong location	Arc depends on knowing where the patch is.
	ordering by the posterior mean does as well as sampling	exploration does not pay within a session; the simpler greedy order is recommended.

The layer is expected to help most where test times differ widely, sessions have

many wrong patches, and the model repeats itself often (Section 4.6).

Chapter 7

Work Plan and Conclusion

7.1 Work Plan

The work takes four months (Table 7.1). Everything before the registration point prepares the experiments; everything after it runs them as planned.

Table 7.1: Work plan.

Month	Work	Milestone
1	Defects4J adapter (checkout, build, faulty methods, coverage, mutants with Major); test runner with per-class timing; the orderings; the simulator	a simulated session runs on one bug
2	Repair loop with feedback and shared caches; modules M1 (Repeat Guard) and M3 (Budget Guard); flakiness checks; development set: model choice, compute estimate, all settings fixed; E1 (2,000 mutants)	**registration**; E1 done
3	E2: 1,200 repair sessions (100 bugs × 3 sessions × 4 orderings, each up to 20 rounds); modules M1 and M3 replayed; E3: 150 sessions (50 bugs × 3 runs)	RQ1–RQ4 analysed
4	Writing; release of the system and data	thesis draft

7.2 Risks

Table 7.2: Main risks and how they are handled.

Risk	Handling
Ordering saves little	the cost breakdown (C1) stands on its own; the result is read as in Chapter 6
Arc only matches a simpler rule	reported as a finding, with the simpler rule recommended
The model service changes	version pinned, answers cached; E1 needs no model
Defects4J is memorised, so sessions are too short	checked on the development set; the model is chosen so that the median session needs at least three rounds
Audits take too long	caps per bug and a fixed order of cuts (Section 5.9)
Results are noisier than assumed	more runs within budget, or the smallest detectable effect is reported
Flaky or order-dependent tests	detected by repeated runs and excluded
Four months is tight	E3 is cut first; E1 and E2 answer RQ1, RQ2 and RQ4

7.3 Threats to Validity

Internal. Costs of every ordering are computed from one audit run in a fixed order; this assumes test results and times do not depend on order, which is checked on a sample. Mutants are not model patches, so E1 only tests the mechanism; every claim about the loop rests on the live runs.

External. One benchmark, one language and one model; the faulty methods are given to the model. Defects4J may be memorised by LLMs [47], which can make sessions shorter; this affects every configuration alike. CI systems that run tests in parallel on many machines are not covered. Two of the practical requirements (P2, P5) rest on indirect evidence, and the give-up report is not tested with developers.

Construct. Success means a plausible patch, not a correct one.

7.4 Future Work

A Python benchmark. The first extension is to Python, using BugsInPy [48], which is modelled on Defects4J: 493 real bugs from 17 Python projects, each with a faulty and a fixed version, the tests that expose the bug, and a command-line tool to check out and run them. Its structure lets the evaluation be repeated with few changes to the modules:

- **Module M1 (Repeat Guard)**: normalisation uses Python's own syntax tree instead of a Java parser;
- **Module M2 (Test Prioritiser)**: the unit of ordering becomes a `pytest` test file, and the start times come from the timings `pytest` reports; the Coverage baseline uses a Python coverage tool;
- **Module M3 (Budget Guard)**: unchanged, since it reads only failure signatures and spending.

Python has no JVM start-up or compilation step, so the share of validation spent on running tests, and with it the room for Module M2, is expected to be larger than on Defects4J; measuring this is itself a test of RQ1. Repository-level tasks such as SWE-bench [22] and contest programs such as ConDefects [21], where a test is one run of the whole program, are later steps.

A stronger model. If time allows, the 20 development bugs are run again with the strongest tier of the same family, `gpt-6-sol` ($2 and $10 per million input and output tokens), in the Default configuration of E2 with three sessions per bug. A stronger model is expected to need fewer rounds and to repeat itself less, which moves the cost towards validating the accepted patch. Comparing its cost breakdown with that of the

main model shows how the room left for each module changes with the strength of the model, a further result for RQ1, and checks that the rule for choosing the model (Section 5.5) does not simply pick a setting in which the wastes exist.

Other directions. Multi-file bugs and imperfect fault localisation; a study of whether the give-up report helps developers; and using the test record to choose which failures to show the model.

7.5 Conclusion

In LLM-based repair, each round costs a model call and a test run, and part of this cost is wasted in three ways. This proposal builds a validation layer with one module aimed at each waste: a Repeat Guard that avoids validating the same patch twice, a Test Prioritiser that uses Arc to reject wrong patches earlier, and a Budget Guard that stops unproductive sessions and reports the reason. It also measures how the cost is distributed, which bounds what each module can save. The layer is evaluated in a controlled simulation on mutated patches and in live runs with a real model on Defects4J, with every outcome's reading fixed in advance.

References

[1] Claire Le Goues, Michael Pradel, and Abhik Roychoudhury. Automated program repair. *Communications of the ACM*, 62(12):56–65, 2019. doi: 10.1145/3318162.

[2] Zichao Qi, Fan Long, Sara Achour, and Martin Rinard. An analysis of patch plausibility and correctness for generate-and-validate patch generation systems. In *Proceedings of the 24th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 24–36, 2015. doi: 10.1145/2771783.2771791.

[3] Chunqiu Steven Xia and Lingming Zhang. Automated program repair via conversation: Fixing 162 out of 337 bugs for $0.42 each using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 819–831, Vienna, Austria, 2024. doi: 10.1145/3650212.3680323.

[4] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. RepairAgent: An autonomous, LLM-based agent for program repair. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pages 2188–2200, Ottawa, ON, Canada, 2025. doi: 10.1109/ICSE55347.2025.00157.

[5] Dávid Hidvégi, Khashayar Etemadi, Sofia Bobadilla, and Martin Monperrus. CigaR: Cost-efficient program repair with LLMs. *arXiv preprint arXiv:2402.06598*, 2024.

[6] Boyang Yang, Zijian Cai, Fengling Liu, Bach Le, Lingming Zhang, Tegawendé F. Bissyandé, Yang Liu, and Haoye Tian. A survey of LLM-based software repair: Taxonomies, design paradigms, and applications. *arXiv preprint arXiv:2506.23749v2*, 2025. Version 2, December 2025; later versions change the title and the number of systems surveyed.

[7] Yannic Noller, Ridwan Shariffdeen, Xiang Gao, and Abhik Roychoudhury. Trust enhancement issues in program repair. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, pages 2228–2240, 2022. doi: 10.1145/3510003.3510040.

[8] Aolin Chen, Haojun Wu, Qi Xin, Steven P. Reiss, and Jifeng Xuan. Studying and understanding the effectiveness and failures of conversational LLM-based repair. In *2025 IEEE/ACM International Workshop on Automated Program Repair (APR)*, pages 56–59, Ottawa, ON, Canada, 2025. doi: 10.1109/APR66717.2025.00014. arXiv:2503.15050.

[9] Yuhua Qi, Xiaoguang Mao, and Yan Lei. Efficient automated program repair through fault-recorded testing prioritization. In *Proceedings of the 29th IEEE International Conference on Software Maintenance (ICSM)*, pages 180–189, Eindhoven, The Netherlands, 2013. doi: 10.1109/ICSM.2013.29.

[10] Zhiyu Fan, Kirill Vasilevski, Dayi Lin, Boyuan Chen, Yihao Chen, Zhiqing Zhong, Jie M. Zhang, Pinjia He, and Ahmed E. Hassan. SWE-Effi: Re-evaluating software AI agent system effectiveness under resource constraints. *arXiv preprint arXiv:2509.09853*, 2025.

[11] Henry Coles, Thomas Laurent, Christopher Henard, Mike Papadakis, and Anthony Ventresque. PIT: A practical mutation testing tool for Java (demo). In *Proceedings of the 25th International Symposium on Software Testing and Analysis (ISSTA)*, pages 449–452, 2016. doi: 10.1145/2931037.2948707.

[12] René Just, Darioush Jalali, and Michael D. Ernst. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 23rd International Symposium on Software Testing and Analysis (ISSTA)*, pages 437–440, 2014. doi: 10.1145/2610384.2628055.

[13] Lingchao Chen, Yicheng Ouyang, and Lingming Zhang. Fast and precise on-the-fly patch validation for all. In *Proceedings of the IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pages 1123–1134, Madrid, Spain, 2021. doi: 10.1109/ICSE43902.2021.00104.

[14] Yuan-An Xiao, Chenyang Yang, Bo Wang, and Yingfei Xiong. Accelerating patch validation for program repair with interception-based execution scheduling. *IEEE Transactions on Software Engineering*, 50(3):618–635, 2024. doi: 10.1109/TSE.2024.3359969. arXiv:2305.03955.

[15] L. G. Mitten. An analytic solution to the least cost testing sequence problem. *Journal of Industrial Engineering*, 11(1):17, 1960.

[16] Herbert A. Simon and Joseph B. Kadane. Optimal problem-solving search: All-or-none solutions. *Artificial Intelligence*, 6(3):235–247, 1975. doi: 10.1016/0004-3702(75)90002-8.

[17] Gregg Rothermel, Roland H. Untch, Chengyun Chu, and Mary Jean Harrold. Prioritizing test cases for regression testing. *IEEE Transactions on Software Engineering*, 27(10):929–948, 2001. doi: 10.1109/32.962562.

[18] Yiling Lou, Jun Yang, Samuel Benton, Dan Hao, Lin Tan, Zhenpeng Chen, Lu Zhang, and Lingming Zhang. When automated program repair meets regression testing—an extensive study on two million patches. *ACM Transactions on Software Engineering and Methodology*, 33(7):180:1–180:23, 2024. doi: 10.1145/3672450.

[19] Sebastian Elbaum, Alexey Malishevsky, and Gregg Rothermel. Incorporating varying test costs and fault severities into test case prioritization. In *Proceedings of the 23rd International Conference on Software Engineering (ICSE)*, pages 329–338, Toronto, ON, Canada, 2001. doi: 10.1109/ICSE.2001.919106.

[20] Branislav Kveton, Csaba Szepesvári, Zheng Wen, and Azin Ashkan. Cascading bandits: Learning to rank in the cascade model. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, volume 37 of *PMLR*, pages 767–776, 2015.

[21] Yonghao Wu, Zheng Li, Jie M. Zhang, and Yong Liu. ConDefects: A complementary dataset to address the data leakage concern for LLM-based fault localization and program repair. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE)*, pages 642–646, 2024. doi: 10.1145/3663529.3663815.

[22] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.

[23] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024.

[24] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 1592–1604, 2024. doi: 10.1145/3650212.3680384.

[25] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Demystifying LLM-based software engineering agents. *Proceedings of the ACM on Software Engineering*, 2(FSE):801–824, 2025. doi: 10.1145/3715754.

[26] Chandra Maddila et al. Agentic program repair from test failures at scale. arXiv:2507.18755, 2025.

[27] OpenHands contributors. OpenHands software agent SDK. https://github.com/OpenHands/software-agent-sdk, commit e656944, 2026.

[28] Aider contributors. Aider: AI pair programming in your terminal. https://github.com/Aider-AI/aider, commit 5dc9490, 2026.

[29] Alexandru Marginean, Johannes Bader, Satish Chandra, Mark Harman, Yue Jia, Ke Mao, Alexander Mols, and Andrew Scott. SapFix: Automated end-to-end

repair at scale. In *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 269–278, 2019. doi: 10.1109/ICSE-SEIP.2019.00039.

[30] Pat Rondon, Renyao Wei, José Cambronero, Jürgen Cito, Aaron Sun, Siddhant Sanyam, Michele Tufano, and Satish Chandra. Evaluating agent-based program repair at Google. In *Proceedings of the 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 365–376, 2025. doi: 10.1109/ICSE-SEIP66354.2025.00038.

[31] Zhihao Lin, Junhua Zhu, Mingyi Zhou, Xin Wang, Zhensu Sun, Renyu Yang, David Lo, and Li Li. To run or not to run: Analyzing the cost-effectiveness of code execution in LLM-based program repair. *Proceedings of the ACM on Software Engineering*, 3(ISSTA):503–525, 2026. doi: 10.1145/3832113. arXiv:2606.26978.

[32] Sergey Mechtaev, Xiang Gao, Shin Hwei Tan, and Abhik Roychoudhury. Test-equivalence analysis for automatic patch generation. *ACM Transactions on Software Engineering and Methodology*, 27(4):15:1–15:37, 2018. doi: 10.1145/3241980.

[33] Lingming Zhang, Darko Marinov, and Sarfraz Khurshid. Faster mutation testing inspired by test prioritization and reduction. In *Proceedings of the 2013 International Symposium on Software Testing and Analysis (ISSTA)*, pages 235–245, 2013. doi: 10.1145/2483760.2483782.

[34] Helge Spieker, Arnaud Gotlieb, Dusica Marijan, and Morten Mossige. Reinforcement learning for automatic test case prioritization and selection in continuous integration. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 12–22, 2017. doi: 10.1145/3092703.3092709.

[35] Jackson A. Prado Lima and Silvia R. Vergilio. A multi-armed bandit approach for test case prioritization in continuous integration environments. *IEEE Transactions on Software Engineering*, 48(2):453–465, 2022. doi: 10.1109/TSE.2020.2992428.

[36] Zixin Zhong, Wang Chi Cheung, and Vincent Y. F. Tan. Thompson sampling algorithms for cascading bandits. *Journal of Machine Learning Research*, 22(218): 1–66, 2021.

[37] Ruida Zhou, Chao Gan, Jing Yang, and Cong Shen. Cost-aware cascading bandits. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 3228–3234, 2018. doi: 10.24963/ijcai.2018/448.

[38] Milos Gligoric, Lamyaa Eloussi, and Darko Marinov. Practical regression test selection with dynamic file dependencies. In *Proceedings of the 2015 International Symposium on Software Testing and Analysis (ISSTA)*, pages 211–222, 2015. doi: 10.1145/2771783.2771784.

[39] Serkan Kirbas, Etienne Windels, Olayori McBello, Kevin Kells, Matthew Pagano, Rafal Szalanski, Vesna Nowack, Emily Rowan Winter, Steve Counsell, David Bowes, Tracy Hall, Sæmundur Haraldsson, and John Woodward. On the introduction of automatic program repair in Bloomberg. *IEEE Software*, 38(4):43–51, 2021. doi: 10.1109/MS.2021.3071086.

[40] Emily Winter, David Bowes, Steve Counsell, Tracy Hall, Sæmundur Haraldsson, Vesna Nowack, and John Woodward. How do developers really feel about bug fixing? Directions for automatic program repair. *IEEE Transactions on Software Engineering*, 49(4):1823–1841, 2023. doi: 10.1109/TSE.2022.3194188.

[41] Emily Rowan Winter, Vesna Nowack, David Bowes, Steve Counsell, Tracy Hall, Sæmundur Haraldsson, John Woodward, Serkan Kirbas, Etienne Windels, Olayori McBello, Abdurahman Atakishiyev, Kevin Kells, and Matthew Pagano. Towards developer-centered automatic program repair: Findings from Bloomberg. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 1578–1588, 2022. doi: 10.1145/3540250.3558953.

[42] William R. Thompson. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, 25(3/4):285–294, 1933. doi: 10.2307/2332286.

[43] Kui Liu, Anil Koyuncu, Tegawendé F. Bissyandé, Dongsun Kim, Jacques Klein, and Yves Le Traon. You cannot fix what you cannot find! An investigation of fault localization bias in benchmarking automated program repair systems. In *Proceedings of the 12th IEEE International Conference on Software Testing, Verification and Validation (ICST)*, pages 102–113, 2019. doi: 10.1109/ICST.2019.00020.

[44] Hyunsook Do and Gregg Rothermel. On the use of mutation faults in empirical assessments of test case prioritization techniques. *IEEE Transactions on Software Engineering*, 32(9):733–752, 2006. doi: 10.1109/TSE.2006.92.

[45] Ali Ghanbari, Samuel Benton, and Lingming Zhang. Practical program repair via bytecode mutation. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 19–30, 2019. doi: 10.1145/3293882.3330559.

[46] René Just. The Major mutation framework: Efficient and scalable mutation analysis for Java. In *Proceedings of the 23rd International Symposium on Software Testing and Analysis (ISSTA)*, pages 433–436, 2014. doi: 10.1145/2610384.2628053.

[47] Daniel Ramos, Claudia Mamede, Kush Jain, Paulo Canelas, Catarina Gamboa, and Claire Le Goues. Are large language models memorizing bug benchmarks? *arXiv preprint arXiv:2411.13323*, 2024.

[48] Ratnadira Widyasari, Sheng Qin Sim, Camellia Lok, Haodi Qi, Jack Phan, Qijin Tay, Constance Tan, Fiona Wee, Jodie Ethelda Tan, Yuheng Yieh, Brian Goh, Ferdian Thung, Hong Jin Kang, Thong Hoang, David Lo, and Eng Lieh Ouh. BugsInPy: A database of existing bugs in Python programs to enable controlled testing and debugging studies. In *Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 1556–1560, 2020. doi: 10.1145/3368089.3417943.